Reviewer Pack — Euler Characteristic of Matroidal Covers and DPLL Refutation…

Entry 852c22751dcc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 06:24:07 UTC

Euler Characteristic of Matroidal Covers and DPLL Refutation Tree Width

Entry ID: `852c22751dcc` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 06:24:07 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Complexity Theory: DPLL refutation tree width

Statement:

> {'text': 'For every 3-CNF formula F, there exists a matroidal cover M of F such that the Euler characteristic χ(M) is upper bounded by a function of the DPLL refutation tree width w(F), specifically χ(M) ≤ c * w(F)^2 for some absolute constant c.', 'equation': 'χ(M) ≤ c * w(F)^2'}

Rationale (proposer's reasoning):

> {'text': "Matroid theory provides structural insights into combinatorial objects, and a matroidal cover of a 3-CNF formula could offer a refined view of the problem's structure. The Euler characteristic is a topological invariant that has been used in complexity analysis, suggesting its potential for capturing the complexity of DPLL refutation tree width."}

Taxonomy category: `MATROIDS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2997078b3243778e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean of χ(M) over all generated 3-CNF formulas F satisfies |χ(M) - c * w(F)^2| ≤ 1 for at least 80% of the seeds, where c is an absolute constant and χ(M) is the Euler characteristic of the matroidal cover M.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `matroid theory AND dpll refutation tree width` - `euler characteristic IN matroidal covers AND complexity theory dpll` - `cnf formula AND upper bound euler characteristic matroid cover dpll`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n, m):
        literals = [f'x{i}' for i in range(1, n+1)] + [f'~x{i}' for i in range(1, n+1)]
        clauses = []
        for _ in range(m):
            clause = random.sample(literals, 3)
            clauses.append(clause)
        return clauses

    def dpll_refutation_tree_width(clauses):
        # Simplified version of DPLL algorithm to estimate tree width
        literals = set()
        for clause in clauses:
            literals.update(clause)
        return len(literals)  # Approximation

    def euler_characteristic(matroid):
        rank = len(matroid)
        n = sum(1 for _ in matroid)
        return rank - n + 1

    def matroidal_cover(clauses):
        matroid = set()
        for clause in clauses:
            matroid.update(clause)
        return matroid
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5):  # Ensure at least 30 instances per seed
            m = random.randint(n, 2*n)
            clauses = generate_3cnf(n, m)
            matroid = matroidal_cover(clauses)
            w_F = dpll_refutation_tree_width(clauses)
            chi_M = euler_characteristic(matroid)

            if chi_M <= 0 or w_F == 0:
                continue

            total_metric_value += chi_M
            instances_tested += 1
            c = Fraction(chi_M, w_F**2)
            if abs(chi_M - c * w_F**2) > 1:
                conjecture_holds = False
                counterexample = f"n={n}, m={m}, chi_M={chi_M}, w_F={w_F}, c={c}"
                break

    return {
        "metric_name": "Euler characteristic",
        "metric_value": total_metric_value / instances_tested if instances_tested > 0 else 0,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjecture_holds': True
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
TRIAL: {'metric_name': 'Euler characteristic', 'metric_value': 1.0, 'instances_tested': 30, 'conjectu
RESULT: SUPPORTED mean=1.0 std=0.0 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The supported verdict is based on a very small sample size of n ≤ 15, which may not be sufficient to establish the validity of the conjecture over all possible instances of 3-CNF formulas. The metric does not scale trivially with n, but the limited number of tests conducted could lead to an overestimation of the general trend.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test results show that the conjecture holds for all tested instances, but the critic challenges the validity of the conclusion due to a small sample size and potential overestimation of the general trend. | next: Further investigation is needed with a larger sample size to confirm or refute the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10591
2	preregistration	ollama_remote	glm4:latest	0	0	5947
3	novelty	ollama_remote	glm4:latest	0	0	4733
4	novelty	ollama_remote	glm4:latest	0	0	6517
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12376
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	87268
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15098
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10157

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	critic	ollama_remote	glm4:latest	0	0	9084
10	judge	ollama_remote	glm4:latest	0	0	10350

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 172120 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/852c22751dcc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/852c22751dcc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/852c22751dcc.tar.gz` (if generated)